

File Roles & Server Mechanics

A combined reference: what every file in the project does, and exactly how a tool call travels from an AI client down to the Ignition Gateway and back.

File Roles — What Every File in This Project Does

A high-level map of every tracked file in `ignition-mcp`, grouped by area.

Entry points & runtime

File	Role
<code>mcp_server.py</code>	The actual entry point. Builds the FastMCP app, defines the <code>lifespan</code> (creates one shared <code>IgnitionClient</code> for the process's whole life), registers all tools, and parses <code>--transport/--host/--port</code> to launch either <code>stdio</code> or <code>streamable-http</code> .
<code>src/ignition_mcp/main.py</code>	Thin <code>python -m ignition_mcp</code> shim — just imports and calls <code>mcp_server.main()</code> . Exists so the package is runnable as a module, not just a script.
<code>run_server.sh</code>	One-line convenience wrapper: <code>uv run python mcp_server.py "\$@"</code> for the streamable-HTTP (default) launch path.

The package (`src/ignition_mcp/`)

File	Role
<code>__init__.py</code>	Package marker, holds <code>__version__</code> .
<code>config.py</code>	The single <code>Settings</code> class (<code>pydantic-settings</code>). Defines every <code>IGNITION_MCP_*</code> environment variable, its default, and its description — this is the source of truth for configuration.

<code>ignition_client.py</code>	The only code that speaks HTTP to the gateway. One method per gateway operation, split into native-REST calls (build a <code>/data/api/v1/...</code> path) and WebDev calls (build a <code>/system/webdev/...</code> URL from a configured endpoint path). Owns auth header selection (API token vs Basic).
<code>tools/__init__.py</code>	<code>register_all(mcp)</code> — imports every tool module and calls its <code>register()</code> function once, wiring all 37 tools into the FastMCP instance. Also documents the full tool inventory in its docstring.
<code>tools/gateway.py</code>	6 tools: gateway info, module health, logs, database/OPC connection status, system metrics. All native REST.
<code>tools/projects.py</code>	8 tools: full project lifecycle — <code>list/get/create/delete/copy/rename/export/import</code> . Native REST.
<code>tools/resources.py</code>	4 tools: read/write individual project resources (Perspective views, scripts, named queries) by path. Native REST.
<code>tools/designers.py</code>	1 tool: list active Designer sessions. Native REST.
<code>tools/tag_providers.py</code>	4 tools: <code>list/get/create/delete</code> tag providers (the containers tags live in). Native REST.
<code>tools/tags.py</code>	9 tools: tag tree browsing (native REST) plus runtime read/write and full tag config CRUD, UDT inspection (all WebDev-backed).
<code>tools/alarms.py</code>	3 tools: active alarms, alarm history, acknowledge. WebDev-backed.
<code>tools/historian.py</code>	1 tool: query historical tag values. WebDev-backed.
<code>tools/execution.py</code>	1 tool: <code>run_gateway_script</code> — arbitrary Python execution on the gateway. Off by default, gated by an explicit env flag; WebDev-backed.

Tests (`tests/`)

File	Role
<code>__init__.py</code>	Package marker for the test suite.
<code>conftest.py</code>	Shared pytest fixtures — a mocked <code>httpx.Response</code> factory, a mocked <code>IgnitionClient</code> , and a fixture that patches all five WebDev endpoint settings to their conventional paths.
<code>test_client.py</code>	Unit tests for <code>IgnitionClient</code> — asserts each method builds the right URL/method/payload, against a mocked httpx client (no real gateway).
<code>test_tools.py</code>	Unit tests for the <code>tools/*.py</code> layer — mocks <code>IgnitionClient</code> itself, checks argument shaping, "not configured" error paths, and exception-to- <code>{"error": ...}</code> wrapping.
<code>test_integration.py</code>	Tests against a <i>real</i> gateway. Skipped unless <code>RUN_LIVE_GATEWAY_TESTS=1</code> is set.

Configuration & secrets

File	Role
<code>.env</code>	Local, untracked (see <code>.gitignore</code>) actual values for every <code>IGNITION_MCP_*</code> setting — gateway URL, API token, WebDev endpoint paths. What the running server actually reads.
<code>.env.example</code>	Template for <code>.env</code> with every variable listed and commented, safe to commit — no real secrets.
<code>.mcp.json</code>	Tells Claude Code how to launch this server as an MCP tool provider for this project (stdio transport, <code>uv run</code> command).
<code>.mcp.json.example</code>	Shows the alternative <code>streamable-http</code> client config (connect to an already-running server by URL) instead of spawning a subprocess.

<code>pyproject.toml</code>	Package metadata, dependencies, and tool config in one place: <code>hatchling</code> build backend, <code>ruff</code> lint rules, <code>mypy</code> strictness, <code>pytest</code> settings (asyncio mode, test paths).
<code>uv.lock</code>	<code>uv</code> 's resolved, pinned dependency lockfile — guarantees everyone installs the exact same versions.

Docs (`docs/`)

File	Role
<code>installation.md</code>	Prerequisites and setup steps (Python version, Ignition version, <code>uv</code> install, first run).
<code>configuration.md</code>	Full walkthrough of every environment variable and precedence rules (env var > <code>.env</code> > default).
<code>webdev-setup.md</code>	The gateway-side deployment guide — the actual Jython <code>doPost</code> scripts to paste into each WebDev resource (<code>tags</code> , <code>tagConfig</code> , <code>alarms</code> , <code>tagHistory</code> , <code>scriptExec</code>), plus security guidance for <code>run_gateway_script</code> .
<code>api-reference.md</code>	Full reference table of all 37 tools: name, transport (native REST vs WebDev), and description.
<code>examples.md</code>	Practical usage examples organized by feature area, written for someone driving the tools through Claude.
<code>troubleshooting.md</code>	Common failure modes and fixes — connection problems, auth errors, WebDev tool errors, Claude Code integration issues.
<code>contributing.md</code>	Dev setup, code style, testing, and PR process for anyone extending the server.

Session-generated documents (this repo root)

File	Role
------	------

<code>ignition_mcp_mechanics.md</code>	Deep-dive on this server's own architecture and request lifecycle, written by tracing the source code directly.
<code>ignition-mcp.md</code>	Chronological summary of every query run in an earlier session, the components involved, and the bugs found/fixed along the way.
<code>ignition_logs.log</code>	A compiled snapshot of gateway log entries from a specific one-hour window, pulled via <code>get_gateway_logs</code> and annotated.
<code>ignition-mcp-workflow.pptx</code>	A slide deck explaining the same architecture — components, MCP protocol, and two worked request traces — built programmatically with <code>python-pptx</code> .
<code>file_roles.md</code>	This file.

Deployment & CI

File	Role
<code>Dockerfile</code>	<code>python:3.11-slim</code> image, installs the package, runs as a non-root user, exposes port 8007, HTTP healthcheck against <code>/mcp</code> . Default command runs the streamable-HTTP transport.
<code>.dockerignore</code>	Excludes git metadata, caches, docs, and dev-only files from the Docker build context.
<code>.github/workflows/ci.yml</code>	On every push/PR: matrix-tests across Python 3.10–3.12 (ruff lint + format check, mypy, pytest), plus a separate job running <code>bandit</code> and <code>safety</code> for security/dependency scanning.
<code>.github/workflows/release.yml</code>	On a <code>v*</code> tag push: builds and pushes a multi-arch (amd64/arm64) Docker image to <code>ghcr.io</code> , then creates a GitHub Release with auto-generated notes.

Misc / tooling

File	Role
<code>README.md</code>	The project's front door — what it is, the 37-tool inventory by category, setup, running instructions, WebDev prerequisites, architecture map, testing.
<code>LICENSE</code>	MIT license text.
<code>.gitignore</code>	Standard Python/IDE/OS ignore patterns, plus project-specific entries (env files, caches).
<code>.ralphrc</code>	Configuration for Ralph , an autonomous-loop runner for Claude Code — call-rate limits, allowed tools, session continuity, and circuit-breaker thresholds for unattended runs.
<code>.claude/settings.local.json</code>	Local, personal Claude Code permission settings for this project — pre-approved Bash command prefixes and MCP tools that don't need a prompt each time, plus which <code>.mcp.json</code> servers are enabled.

ignition-mcp — How It Works

This document explains the project's file layout and the exact mechanics of how a tool call travels from an AI client down to the Ignition Gateway and back.

What this project is

`ignition-mcp` is a **Model Context Protocol (MCP) server** written in Python with [FastMCP](#). It exposes 37 tools that let an AI assistant (Claude Code, Claude Desktop, or any MCP client) inspect and manage an **Ignition Gateway** — projects, tag providers, tag values/config, alarms, tag history, and gateway diagnostics — without the assistant needing to know Ignition's REST API or scripting model directly.

It talks to the gateway over two completely different channels:

- **Native REST API** (`/data/api/v1/...`) — built into Ignition 8.3+, no extra setup required. Covers configuration-level data: gateway info, module health, logs, projects, project resources, designers, tag providers, and (as of the `database-connection/opc-connection` resource types) connection status.
- **WebDev module scripts** (`/system/webdev/...`) — custom Python scripts you must deploy yourself in the Designer. Required for anything the native REST API doesn't cover: live tag reads/writes, tag CRUD, UDTs, alarms, tag history, and arbitrary gateway script execution.

Components & the two backends



Top-level layout

```
ignition-mcp/
├── mcp_server.py      # Entry point — builds the FastMCP app, owns the lifespan
├── pyproject.toml     # Dependencies, ruff/mypy/pytest config
├── run_server.sh      # Convenience wrapper: uv run python mcp_server.py
├── Dockerfile         # python:3.11-slim image, non-root user, HTTP healthcheck
├── .env / .env.example # IGNITION_MCP -prefixed configuration
├── .mcp.json          # How Claude Code launches this server (stdio, via uv)
├── src/ignition_mcp/  # The actual package (see below)
├── tests/             # Unit + integration test suite
└── docs/              # WebDev setup guide, API reference, troubleshooting, etc.
```

src/ignition_mcp/ — the package

```
src/ignition_mcp/
├── __init__.py        # version = "0.1.0"
├── main.py            # python -m ignition_mcp entry, delegates to mcp_server.main()
├── config.py          # Settings — pydantic-settings, env prefix IGNITION_MCP
├── ignition_client.py # IgnitionClient — the only thing that speaks HTTP to the gateway
├── tools/
│   ├── __init__.py    # register_all(mcp) — wires every tool module into FastMCP
│   ├── gateway.py     # get_gateway_info, get_module_health, get_gateway_logs,
│   │                 # get_database_connections, get_opc_connections, get_system_metrics
│   ├── projects.py    # list/get/create/delete/copy/rename/export/import project
│   ├── resources.py   # list/get/set/delete_project_resource (Perspective views, scripts, queries)
│   ├── designers.py   # list_designers
│   ├── tag_providers.py # list/get/create/delete_tag_provider
│   ├── tags.py        # browse_tags, read_tags, write_tag, tag config CRUD, UDT tools
│   ├── alarms.py      # get_active_alarms, get_alarm_history, acknowledge_alarms
│   ├── historian.py     # get_tag_history
│   └── execution.py    # run_gateway_script (off by default)
```

Every tool module follows the exact same three-part shape:

1. A private `_client(ctx)` helper that pulls the shared `IgnitionClient` out of the FastMCP request context.
2. One `async def` per tool — thin wrapper that validates/shapes arguments (via `Annotated[...]`, `Field(...)` for MCP-visible parameter docs), calls the matching `IgnitionClient` method, and catches exceptions into `{"error": "..."}` so a failure is a normal tool result, not a crash.
3. A `register(mcp)` function that calls `mcp.tool()(fn)` for each tool in the module. `tools/__init__.py:register_all()` calls every module's `register()` once, at import time in `mcp_server.py`.

Request lifecycle — from tool call to gateway and back



Key mechanics:

- **One shared `IgnitionClient` per server process.** `mcp_server.py`'s `lifespan()` async context manager creates a single `IgnitionClient` (one `httpx.AsyncClient`) when the server starts, stores it in `{"client": client}`, and every tool call across every request reuses the same instance via `ctx.request_context.lifespan_context["client"]`. It's closed once, on server shutdown.
- **Auth is chosen automatically.** `IgnitionClient._auth_headers()` prefers the API token (X-Ignition-API-Token header) if `IGNITION_MCP_IGNITION_API_KEY` is set; otherwise it falls back to HTTP Basic auth built from `IGNITION_MCP_IGNITION_USERNAME/_PASSWORD`.
- **Errors never crash the tool call.** Every tool function wraps its client call in `try/except Exception` and returns `{"error": "..."}`, so a 404/500/timeout from the gateway comes back to the AI as readable JSON instead of failing the whole MCP request.
- **WebDev-backed tools gate on configuration first.** Tools like `read_tags` or `get_active_alarms` check a `webdev_*_configured` property (a simple `bool(...)` on the relevant `IGNITION_MCP_WEBDEV_*_ENDPOINT` setting) *before* trying the call, returning a structured "not configured" error with setup guidance if the endpoint isn't set.

Two backends, one client class

IgnitionClient (in `ignition_client.py`) has one method per gateway operation, grouped by backend:

Native REST — methods build a `/data/api/v1/...` path and call `self._request()` directly. Examples: `get_gateway_info` → `/data/api/v1/gateway-info`, `list_projects` → `/data/api/v1/projects/list`, `browse_tags` → `/data/api/v1/entity/browse`, and the generic resource-type listing pattern used by `list_tag_providers`, `get_database_connections`, and `get_opc_connections`: `/data/api/v1/resources/list/{module}/{resource-type}` (e.g. `ignition/tag-provider`, `ignition/database-connection`, `ignition/opc-connection`).

WebDev — methods build a URL from a *configured resource path*, not a fixed route: `_webdev_url()` / `_webdev_tag_config_url()` / `_webdev_alarm_url()` / `_webdev_tag_history_url()` / `_webdev_script_exec_url()` each read their respective `IGNITION_MCP_WEBDEV_*_ENDPOINT` setting and produce `/system/webdev/{that path}`. The request body is always a JSON payload with an `action` field (e.g. `{"action": "getActive", ...}`), because a single WebDev resource multiplexes several operations via its `doPost` handler — see `docs/webdev-setup.md` for the actual gateway-side Python each endpoint expects (e.g. the `tags` endpoint dispatches on whether the body has `paths` (read) or `tagPath` (write); `tagConfig`, `alarms`, and `scriptExec` dispatch on `action`).

This is why WebDev tools require you to hand-deploy Jython scripts in the Designer first (`docs/webdev-setup.md` has the exact code to paste into each WebDev resource's `doPost`), while native-REST tools work out of the box against any Ignition 8.3+ gateway.

Configuration (`config.py`)

A single `pydantic-settings Settings` class, loaded once as the module-level `settings` singleton. All fields read from environment variables prefixed `IGNITION_MCP_`, with `.env` as a fallback file. Key fields:

Field	Purpose
<code>ignition_gateway_url</code>	Base URL, e.g. <code>http://localhost:8088</code>
<code>ignition_api_key</code> / <code>ignition_username</code> / <code>ignition_password</code>	Auth — API key wins if set
<code>ssl_verify</code>	Set false for self-signed gateway certs
<code>webdev_tag_endpoint</code> , <code>webdev_tag_config_endpoint</code> , <code>webdev_alarm_endpoint</code> , <code>webdev_tag_history_endpoint</code> , <code>webdev_script_exec_endpoint</code>	Per-feature WebDev resource paths; empty = feature disabled
<code>enable_script_execution</code>	Hard off-switch for <code>run_gateway_script</code> , independent of whether its WebDev endpoint is configured

`server_host / server_port`

Bind address for `streamable-http` transport (default `127.0.0.1:8007`)

How the server actually starts

`mcp_server.py` is the single entry point (both `run_server.sh` and `python -m ignition_mcp` funnel into its `main()`):

1. `sys.path` gets `src/` prepended so `ignition_mcp` is importable without installation.
2. `mcp = FastMCP("ignition-mcp", lifespan=lifespan)` is constructed at module level, then `register_all(mcp)` registers all 37 tools immediately — this happens once, at import time, regardless of transport.
3. `main()` parses `--transport` (`streamable-http` default, or `stdio`), `--host`, `--port`, and calls `mcp.run(...)` accordingly.

Two ways this project runs in practice:

- **stdio** (what `.mcp.json` uses for Claude Code): the MCP client spawns `uv run --no-sync python mcp_server.py --transport stdio` as a subprocess and talks to it over stdin/stdout. There's no network port — the process lives and dies with the client's connection to it. This is why editing the source code requires a client-side reconnect (`/mcp` → reconnect) to pick up changes: the old subprocess is still running the old code in memory until it's restarted.
- **streamable-http** (default, used by `run_server.sh` and the `Dockerfile`): binds `host:port` (`127.0.0.1:8007` locally, `0.0.0.0:8007` in the container) and serves the MCP protocol over HTTP at `/mcp`. Multiple clients could connect to the same running server.

Tests (`tests/`)

- `test_client.py` — unit tests for `IgnitionClient` methods against a mocked `httpx.AsyncClient` (via the `mock_client/mock_httpx_response` fixtures in `conftest.py`). Asserts the right URL/method/payload is built for each method.
- `test_tools.py` — unit tests for the tool-layer functions in `tools/*.py`, mocking `IgnitionClient` itself to check argument shaping and error handling.
- `test_integration.py` — exercises tools against a *real* gateway; skipped unless `RUN_LIVE_GATEWAY_TESTS=1` is set.
- `conftest.py` also provides a `webdev_settings` fixture that monkeypatches all five WebDev endpoint settings to their conventional `Global/GatewayAPI/*` paths, for tests that need WebDev-backed tools to appear "configured".

Known gap fixed in this session

The native-REST `get_database_connections/get_opc_connections` methods originally called `/data/api/v1/connections/database` and `/data/api/v1/connections/opc`, which 404 on Ignition 8.3.8 — those routes don't exist. The correct routes follow the same generic resource-list pattern already used by `list_tag_providers`: `/data/api/v1/resources/list/ignition/database-connection` and `/data/api/v1/resources/list/ignition/opc-connection`. Fixed in commit `33bfc92`. Note there's a third, related concept

not exposed by any tool yet: OPC UA **device connections** (PLC drivers) live under a different resource type, [/data/api/v1/resources/list/com.inductiveautomation.opcua/device](#) — distinct from the [ignition/opc-connection](#) resource type, which is the OPC UA *server* connection (e.g. the loopback connection to Ignition's own OPC UA server).